
Hoare Logic

LN chapter 5, 6.1, 6.2

Hoare Logic is used to reason about the correctness of programs. In the end, it reduces a program and its specification to a set of verification conditions.

Overview

- Hoare logic (Ch 5, 6.1)
 - Hoare triple
 - Rules for basic statements // SEQ, IF, ASG
 - Weakest pre-condition
 - Example
- Weakest pre-condition (LN 6.2)

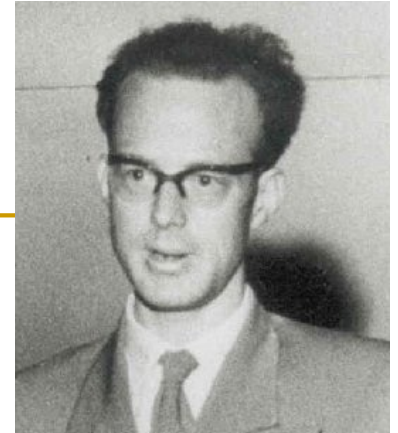
Hoare logic basics

LN Chapter 5, 6.1

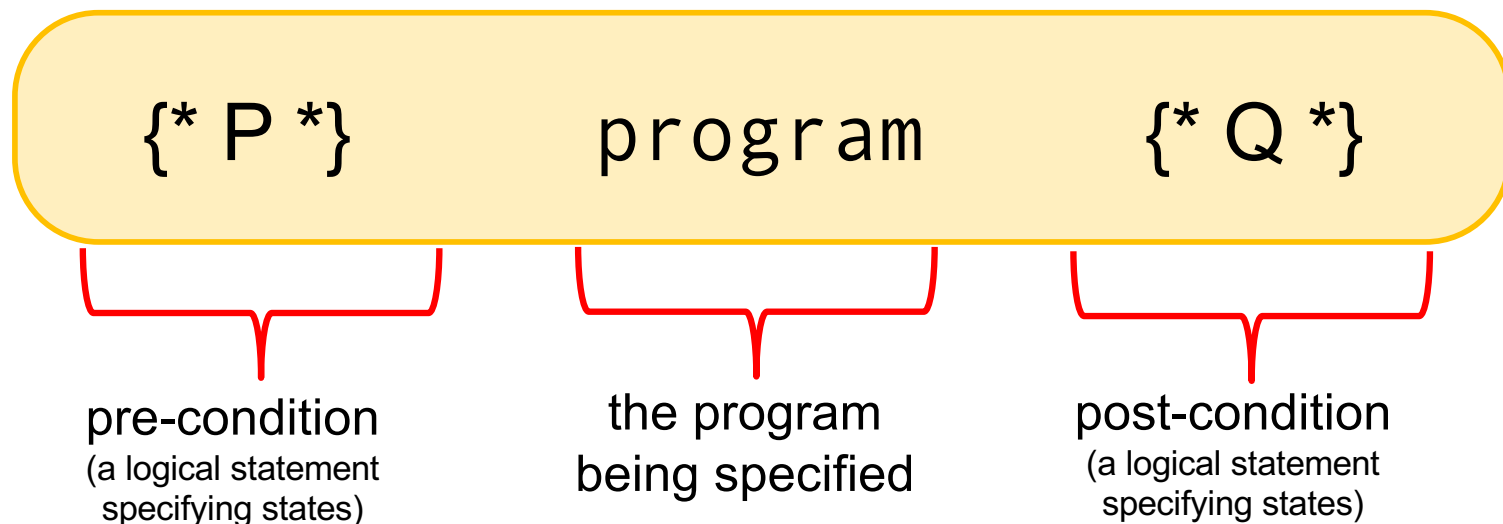
Why Hoare logic?

- We've looked at proofs about **purely functional** programs
 - e.g., `rev` and `rv` are purely functional
 - $\forall s :: \text{rev } s = \text{rv } [] \ s$
 - **Purely functional** programs only take in an input and return an output; the behavior is not affected by anything else and it does not affect anything else
 - Thus, we prove properties about it mathematically, as above
- Now we reason about **stateful**/imperative/object oriented programs
 - These are affected by and affect the state of the memory, etc.

Hoare logic



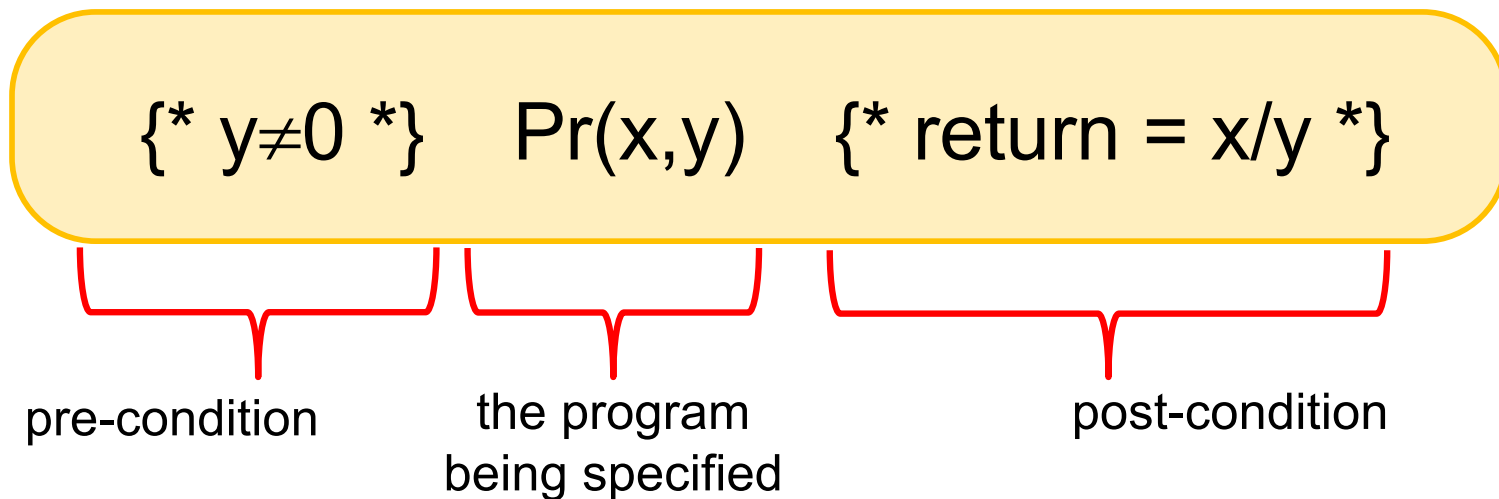
- Invented by Tony Hoare (1934-2026) in 1969
 - Influenced by Robert Floyd
- The **Hoare triple** expresses the effect the program has on state



- The triple says: if we are in a state satisfying P , after the program runs, we are in a state satisfying Q .

Hoare triple

- Example:



- Meaning: if we are in a state where $y \neq 0$, then after running $\text{Pr}(x, y)$, we are in a state where $\text{return} = x/y$.

Partial and total correctness interpretation

$$\{^* P ^*\} \quad \text{Pr}(x) \quad \{^* Q ^*\}$$

- **Total correctness interpretation** of Hoare triple: if the program is executed on a state satisfying P , it will **terminate** in a state satisfying Q .
- **Partial correctness interpretation**: if the program is executed on a state satisfying P , and **if it terminates**, it will terminate in a state satisfying Q .
- Partial correctness is of course weaker, but on the other hand easier to prove. Useful in situations where we can afford to postpone concerns about termination.

Limitations of Hoare logic

- Not every type of behavioral property can (naturally) be captured with Hoare logic.
- Examples:
 - If we want to reason about the order in which certain things happen when a program is run.
 - Alternative: CSP (communicating sequential process)
 - If we want to reason about how often a state is visited.
 - Alternative: temporal logic
 - If we want to reason about trusting information.
 - Alternative: logic of belief.

Hoare Logic

Components

- Basic statement is the Hoare triple
 - $\{*P*\} \text{ program } \{*Q*\}$
- To prove a specific statement, we use **inference rules**, like the one below
 - It says that if the statements above the line hold, then we can conclude the statements below the line

$$\frac{\{*P*\} \ S \ \{*Q*\} \qquad \{*Q*\} \ T \ \{*R*\}}{\{*P*\} \ S \ ; \ T \ \{*R*\}}$$

Sequential decomposition

- The first rule is the most important

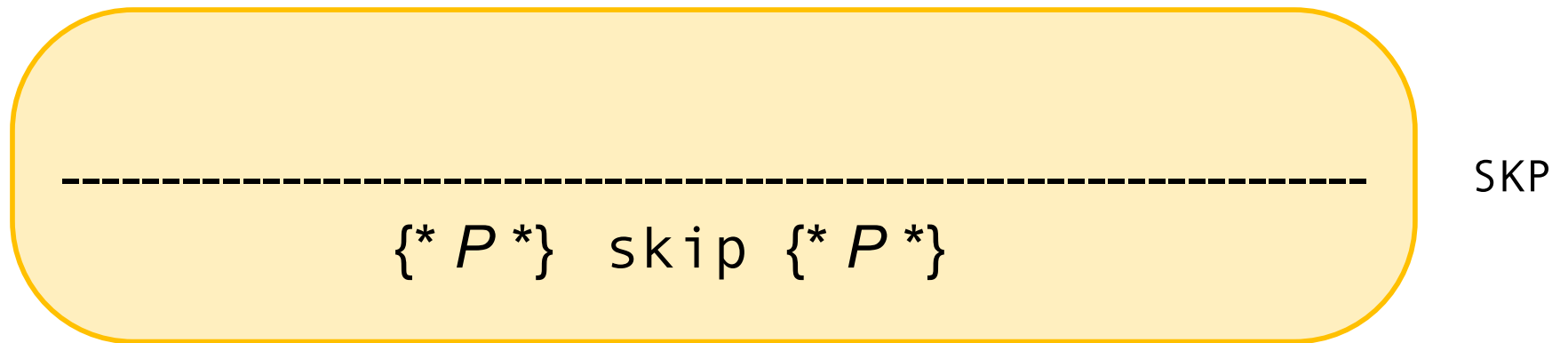
$$\frac{\begin{array}{cc} \{ * P * \} \quad S \quad \{ * Q * \} & \{ * Q * \} \quad T \quad \{ * R * \} \end{array}}{\{ * P * \} \quad S \quad ; \quad T \quad \{ * R * \}}$$

SEQ
(B.1.6)

- It tells us that if we want to reason about a program consisting of two statements S and T in sequence, then we can do this by reasoning about the programs S and T separately
- Thus, we break down a program into its steps and reason one-by-one

Skip

- Here is another basic rule



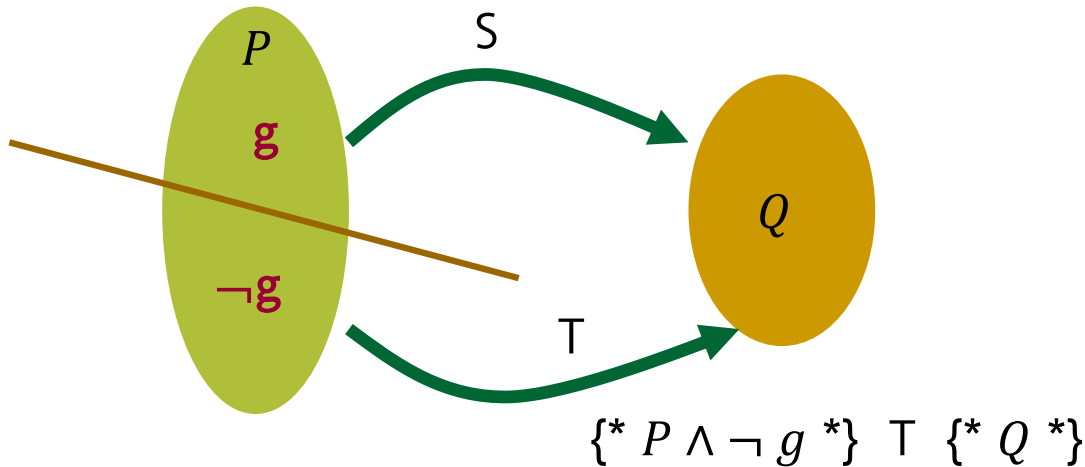
- If P holds before executing `skip`, then it still holds afterwards

Inference rule for if

$$\{ * P \wedge g * \} \quad S \quad \{ * Q * \}$$
$$\{ * P \wedge \neg g * \} \quad T \quad \{ * Q * \}$$

$$\{ * P * \} \text{ if } g \text{ then } S \text{ else } T \{ * Q * \}$$

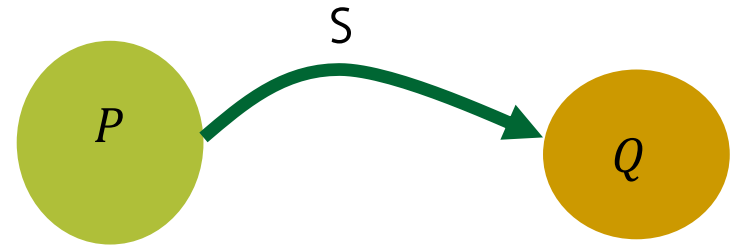
IF
(B.1.5)

$$\{ * P \wedge g * \} \quad S \quad \{ * Q * \}$$


Mental model:

- Sometimes think of P as a **logical statement**
- Sometimes think of P as the **set of all states satisfying P**

Predicates as subsets



- Let W denote the set of all possible states.
- Let $|P|$ denote $\{s \in W \mid P s\}$, that is the set of all states where P holds.
 - This gives a bijection between subsets $|P|$ and predicates P , meaning that we are justified in switching back and forth between the mental representations.
 - We usually let P denote $|P|$, unless it's confusing.
- If our program is deterministic, a program T gives rise to a function
$$T : W \rightarrow W$$
- When we draw a picture as above, we illustrate the fact that
$$\{ * P * \} \quad T \quad \{ * Q * \}$$
or, equivalently, that
$$T(P) \subseteq Q$$
i.e., that T takes the states satisfying P into the states satisfying Q .

Inference rule for assignment

??

$\{^* P ^*\} \quad x := e \quad \{^* Q ^*\}$

- **Idea:** let's first construct a pre-condition W that holds before the assignment **if and only if** Q holds after the assignment.
- Then we can equivalently prove $P \Rightarrow W$

Examples

- $\{^* W? ^*\} \quad x := 10 \quad \{^* x = y ^*\}$
 $W: 10 = y$
- $\{^* W? ^*\} \quad x := x + a \quad \{^* x = y ^*\}$
 $W: x + a = y$
- $\{^* W? ^*\} \quad x := x + 1 \quad \{^* x > 10 ^*\}$
 $W: x + 1 > 10$
- Generally: set $W := Q[e/x]$
 - $Q[e/x]$ is the statement obtained by replacing all free occurrences with x by e

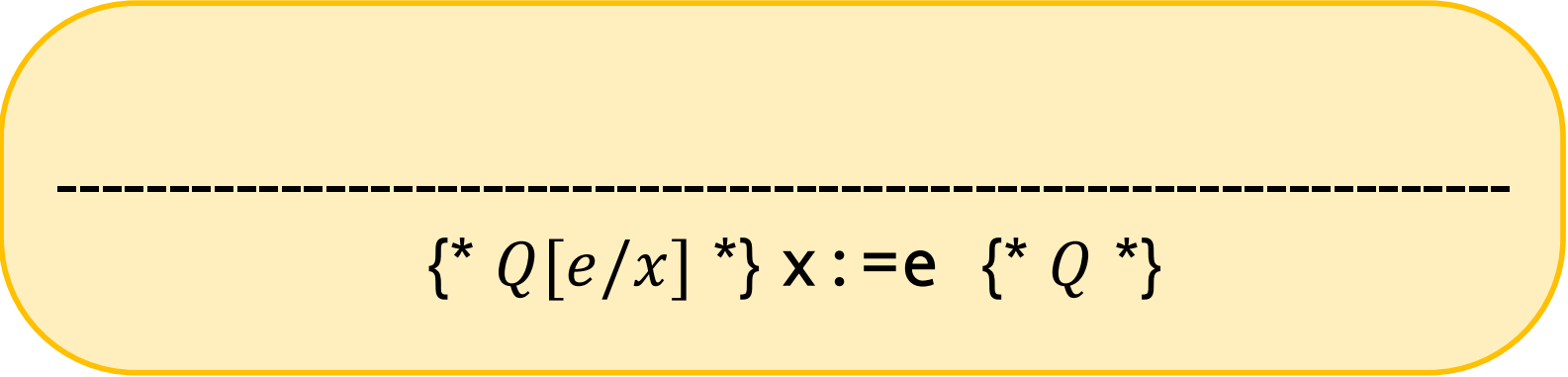
Assignment

- Theorem:

Q holds after $x := e$

if and only if

$Q[e/x]$ held before $x := e$.



$\{^* Q[e/x] ^*\} x := e \{^* Q ^*\}$

ASG

More basic rules

■ Pre-condition strengthening:

$$\vdash P \Rightarrow Q \quad , \quad \{ * Q * \} \quad S \quad \{ * R * \}$$

$$\{ * P * \} \quad S \quad \{ * R * \}$$

PRE
(B.1.2)

■ Post-condition weakening:

$$\{ * P * \} \quad S \quad \{ * Q * \} \quad , \quad \vdash Q \Rightarrow R$$

$$\{ * P * \} \quad S \quad \{ * R * \}$$

POST
(B.1.1)

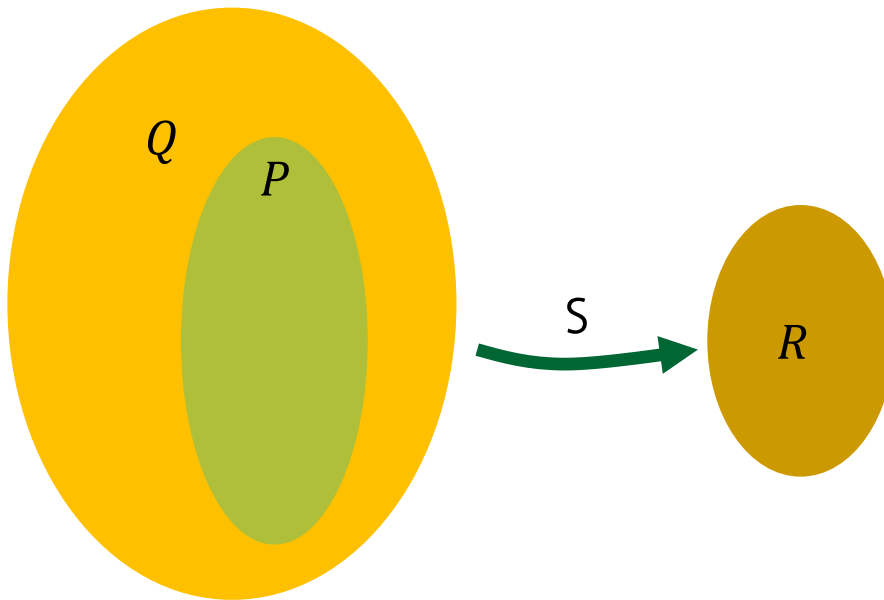
Pre-condition strengthening picture

- **Pre-condition strengthening:**

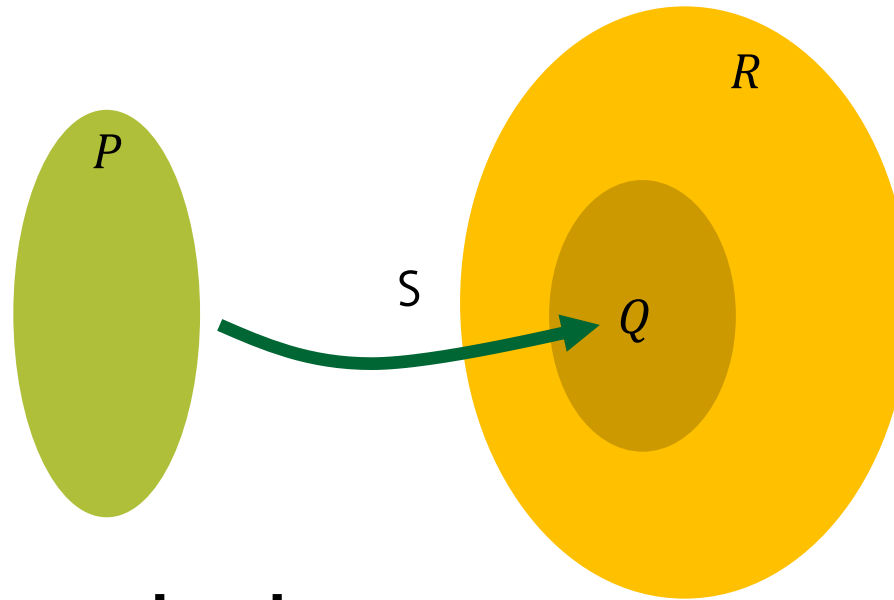
$$\vdash P \Rightarrow Q \quad , \quad \{^* Q ^*\} S \{^* R ^*\}$$

$$\{^* P ^*\} S \{^* R ^*\}$$

PRE
(B.1.2)



Post-condition weakening picture



■ Post-condition weakening:

$$\{^* P ^*\} \ S \ \{^* Q ^*\} , \ \vdash \ Q \Rightarrow R$$

$$\{^* P ^*\} \ S \ \{^* R ^*\}$$

POST
(B.1.1)

PRE + ASG

$$\frac{\begin{array}{c} \text{----- ASG} \\ \vdash P \Rightarrow Q[e/x] \quad \{^* Q[e/x]^*\} x := e \{^* Q^*\} \\ \text{----- PRE} \end{array}}{\{^* P^*\} x := e \{^* Q^*\}}$$

is a proof of

$$\frac{\vdash P \Rightarrow Q[e/x]}{\{^* P^*\} x := e \{^* Q^*\}}$$

More basic rules

- **Conjunction of Hoare triples:**

$$\frac{\{^* P_1 ^*\} \ S \ \{^* Q_1 ^*\} \qquad \{^* P_2 ^*\} \ S \ \{^* Q_2 ^*\}}{\{^* P_1 \wedge P_2 ^*\} \ S \ \{^* Q_1 \wedge Q_2 ^*\}}$$

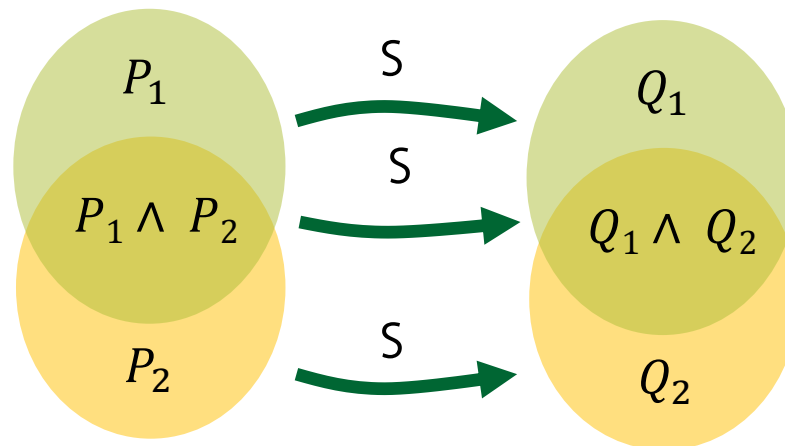
- **Disjunction of Hoare triples**

$$\frac{\{^* P_1 ^*\} \ S \ \{^* Q_1 ^*\} \qquad \{^* P_2 ^*\} \ S \ \{^* Q_2 ^*\}}{\{^* P_1 \vee P_2 ^*\} \ S \ \{^* Q_1 \vee Q_2 ^*\}}$$

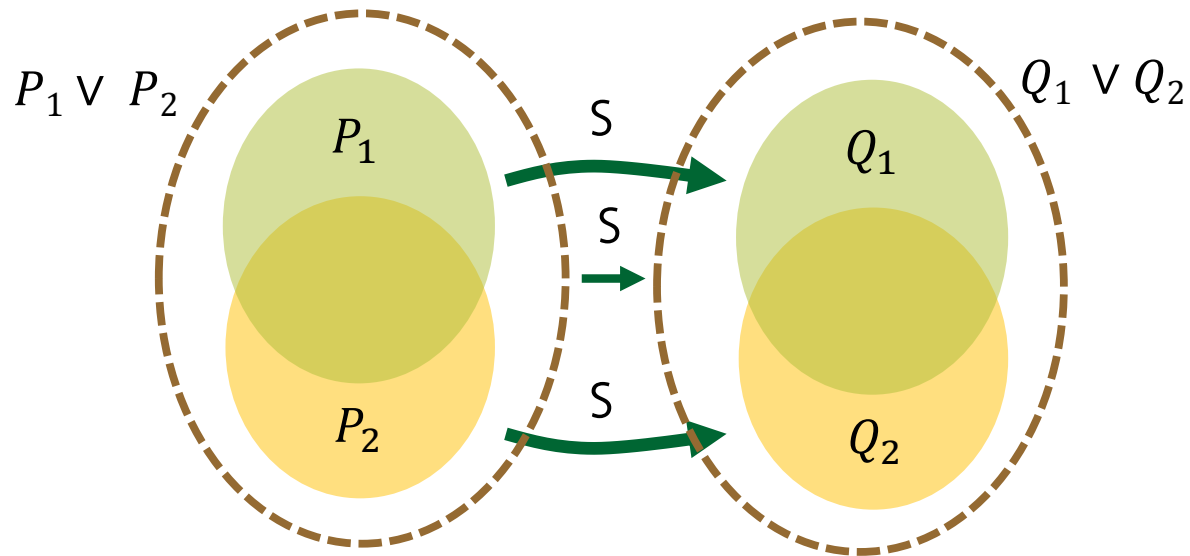
Conjunction picture

- **Conjunction of Hoare triples:**

$$\frac{\{^* P_1 ^*\} \quad S \quad \{^* Q_1 ^*\} \qquad \{^* P_2 ^*\} \quad S \quad \{^* Q_2 ^*\}}{\{^* P_1 \wedge P_2 ^*\} \quad S \quad \{^* Q_1 \wedge Q_2 ^*\}}$$



Disjunction picture



■ Disjunction of Hoare triples

$$\begin{array}{c} \{^* P_1 ^*\} \quad S \quad \{^* Q_1 ^*\} \qquad \{^* P_2 ^*\} \quad S \quad \{^* Q_2 ^*\} \\ \hline \{^* P_1 \vee P_2 ^*\} \quad S \quad \{^* Q_1 \vee Q_2 ^*\} \end{array}$$

Weakest pre-condition

LN Section 6.2

How does a proof proceed now ?

- Example:

$\{^* x \neq y ^*\} \quad \text{tmp} := x \ ; \ x := y \ ; \ y := \text{tmp} \quad \{^* x \neq y ^*\}$

- To use the SEQ rule you need intermediate assertions:

$\{^* x \neq y ^*\} \quad \text{tmp} := x \quad \{^* ? ^*\} \ ; \ x := y \quad \{^* ? ^*\} \ ; \ y := \text{tmp} \quad \{^* x \neq y ^*\}$

- Inference rules of Hoare logic tell us how to prove the claim once we have intermediate assertions
- But they **do not** tell us how to come up with these intermediate assertions in the first place.

Weakest pre-condition

- We want a function

$$wp : \text{Stmt} \rightarrow \text{Pred} \rightarrow \text{Pred}$$

- that takes a program statement and a predicate on states
- and returns a predicate on states
such that
- $\{^* wp\ S\ Q\ ^*\} S \{^* Q\ ^*\}$
- and $wp\ S\ Q$ is the **weakest** pre-condition making the above hold.
- Then if $\{^* P\ ^*\} S \{^* Q\ ^*\}$ held, we would get $P \Rightarrow wp\ S\ Q$.

Weakest pre-condition

- So we characterize **wp** as follows (Def. 6.2.3):

$$\{^* P ^*\} S \{^* Q ^*\} \quad \equiv \quad P \Rightarrow \mathbf{wp} S Q$$

- *Corollary-1*: Such a **wp** is a valid pre-condition. That is, you can prove:

$$\{^* \mathbf{wp} S Q ^*\} S \{^* Q ^*\}$$

- *Corollary-2* : **wp** reduces program verification problems to proving implications, for which we already have a “tool” for (Predicate Logic).

Weakest pre-condition

- Again, the characterization of wp:

$$\{^* P ^*\} S \{^* Q ^*\} \quad \equiv \quad P \Rightarrow \text{wp } S Q$$

- *Corollary-3*: the reduction is complete. That is, if either side above is not valid, neither is the other side.
- In particular: a counterexample invalidating $P \Rightarrow \text{wp } S Q$ is a state satisfying P that after running S produces a state that is not in Q . Such an input is useful for debugging S .

But...

- But this characterization is **not constructive**:

$$\{^* P ^*\} S \{^* Q ^*\} \equiv P \Rightarrow \text{wp } S Q$$

- That is, it does not tell us how to actually calculate **wp** $S Q$.
- To be actually usable, we need to come up with a constructive definition for wp.

Some notes before we attempt to define wp

- This is the meta property we want to have:

$$P \Rightarrow \text{wp } S \ Q \quad \equiv \quad \{^* P ^*\} S \ \{^* Q ^*\}$$

- A definition/implementation of **wp** should be at least “**sound**”:

$$P \Rightarrow \text{wp } S \ Q \quad \Rightarrow \quad \{^* P ^*\} S \ \{^* Q ^*\}$$

- It should also be “**complete**”:

$$P \Rightarrow \text{wp } S \ Q \quad \Leftarrow \quad \{^* P ^*\} S \ \{^* Q ^*\}$$

Or, Equivalently:

$$\cancel{P \Rightarrow \text{wp } S \ Q} \quad \Rightarrow \quad \cancel{\{^* P ^*\} S \ \{^* Q ^*\}}$$

Subset view of wp

- This is the meta property we want to have:

$$P \Rightarrow \text{wp } S Q \equiv \{^* P ^*\} S \{^* Q ^*\}$$

- In terms of subsets, this is equivalently stated as:

$$P \subseteq \text{wp } S Q \equiv S(P) \subseteq Q$$

But notice that the following is true (from logic or basic set theory)

$$P \subseteq S^{-1}(Q) \equiv S(P) \subseteq Q$$

Therefore, $S^{-1}(Q)$ has the property we want!

$$\text{wp } S Q = S^{-1}(Q)$$

- It also matches our intuition: $\text{wp } S Q$ is the collection of all states that are sent to Q by S .

Weakest pre-condition of skip and assignment

$$\text{wp } \text{skip } Q = Q$$

Proof: As a function, **skip** is the identity. Therefore, $\text{skip}^{-1}(Q) = \text{id}^{-1}(Q) = Q$.

$$\text{wp } (x:=e) Q = Q[e/x]$$

Proof: We want to show $(x:=e)^{-1}(Q) = Q[e/x]$. For any state s , we have

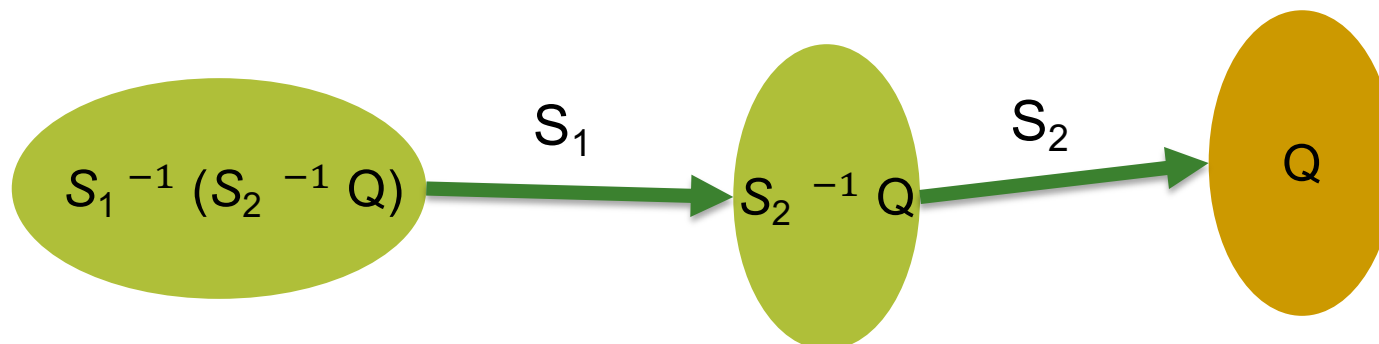
$$\begin{aligned} & s \in (x:=e)^{-1}(Q) \\ = & (x:=e) s \in Q \\ = & (x:=e) s \text{ satisfies } Q \\ = & s \text{ satisfies } Q[e/x] \\ = & s \in Q[e/x]. \end{aligned}$$

wp of SEQ

$$\text{wp } (S_1 ; S_2) \ Q = \text{wp } S_1 \ (\text{wp } S_2 \ Q)$$

Proof: We have that

$$\begin{aligned} & \text{wp } (S_1 ; S_2) \ Q \\ &= (S_1 ; S_2)^{-1} \ Q \\ &= (S_2 \circ S_1)^{-1} \ Q \\ &= S_1^{-1} (S_2^{-1} \ Q) \\ &= \text{wp } S_1 \ (\text{wp } S_2 \ Q). \end{aligned}$$



wp of IF

$$\mathbf{wp} \text{ (if } g \text{ then } S_1 \text{ else } S_2) Q = (g \wedge \mathbf{wp} S_1 Q) \vee (\neg g \wedge \mathbf{wp} S_2 Q)$$

Proof: As a function, **if** g **then** S_1 **else** S_2 does the following to a state s :

$$(\text{if } g \text{ then } S_1 \text{ else } S_2) s = \begin{cases} S_1(s) & \text{if } s \in g \\ S_2(s) & \text{o/w} \end{cases}$$

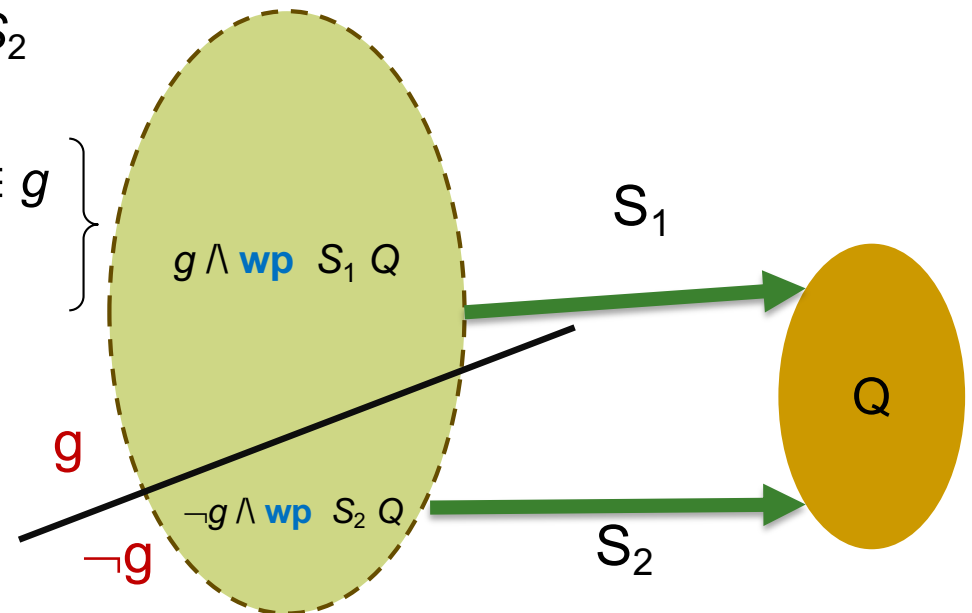
So

$$(\text{if } g \text{ then } S_1 \text{ else } S_2)^{-1} Q$$

$$= \left\{ \begin{array}{ll} S_1 & \text{on } g \\ S_2 & \text{on } \bar{g} \end{array} \right\}^{-1} Q$$

$$= (g \cap S_1^{-1} Q) \cup (\bar{g} \cap S_2^{-1} Q)$$

$$= (g \wedge \mathbf{wp} S_1 Q) \vee (\neg g \wedge \mathbf{wp} S_2 Q).$$



Alternative formulation of wp IF

$$\mathbf{wp} \text{ (if } g \text{ then } S_1 \text{ else } S_2) Q = (g \wedge \mathbf{wp} S_1 Q) \vee (\neg g \wedge \mathbf{wp} S_2 Q)$$

=

$$(g \Rightarrow \mathbf{wp} S_1 Q) \wedge (\neg g \Rightarrow \mathbf{wp} S_2 Q)$$

This formulation for the **wp** of if-then-else is more convenient for working out proofs. E.g. in a deductive proof its conjunctive form would translate to proving two goals.

How does a proof proceed now ?

- $\{ * x \neq y * \}$ tmp := x ; x := y ; y := tmp $\{ * x \neq y * \}$

1. Calculate:

$$W = \text{wp} \quad (\text{tmp} := x ; x := y ; y := \text{tmp}) \quad x \neq y$$

2. Then prove: $x \neq y \Rightarrow W$

- We calculate the intermediate assertions, rather than manually figuring them out!

Example

$\{^* 0 \leq i \wedge \neg \text{found} \wedge (\text{found} = (\exists k : 0 \leq k < i : a[k]=x)) \}^*$

$\text{found} := a[i]=x ;$

$(a[i]=x) = (\exists k : 0 \leq k < i+1 : a[k]=x)$

$i := i+1$

$\text{found} = (\exists k : 0 \leq k < i+1 : a[k]=x)$

$\{^* \text{found} = (\exists k : 0 \leq k < i : a[k]=x) \}^*$

$\text{wp } (x:=e) Q = Q[e/x]$

Some notes about the verification approach

- In our training, we prove $P \Rightarrow W$ by hand.
- In practice, to some degree this can be automatically checked using e.g. a SAT solver. With a SAT solver we instead check if:

$\neg(P \Rightarrow W)$ is **satisfiable**

If it is, then $P \Rightarrow W$ is **not** valid. Otherwise $P \Rightarrow W$ is valid.

- If the definition of wp is also “complete” (in addition to being “sound”), the witness of the satisfiability of $\neg(P \Rightarrow W)$ is essentially an input that will expose a bug in the program → useful for debugging.